

The Drain Set

continuous band refresh by scheduling rather than by payment

local_mixing — design note

2026-08-24

Branch `ssg-gen-mix-clean`; all changes in `src/preprocessing/gadgets.rs`. The band-turnover mechanism of the swap-refresh gadgetizer is replaced: instead of *paying* to release a band variable before rewriting it, retirements the fold is already making are *steered* at the variable until it comes free on its own. Measured at $n = 128$: $2.3\times$ the band turnover for $+0.3\%$ gates.

1 What changed

A band variable’s value is a **frozen function of the input**, and that is a signature. Rolling relocates a variable between wires, but the wire’s Boolean function is the same function, so the band population separates from the carriers by LIFETIME alone, with no need to guess a wire set. Band width is no defense — the attack recovers the population, not a subset.

The existing answer was the **epoch** channel (`retire_refill`): every few folds, pick a band variable, re-source every live mask that names it, then rewrite the wire. It works, and it is expensive.

	old (epoch)	new (drain set)
band turnovers, $n = 128$	6.32	14.72
gates	884,408	886,695 (+0.3%)
forward / reverse verify	pass	pass

Frequent refresh is the point: a band value that changes many times during the computation is never a stable function of the input for long enough to be tracked, and the intervals over which it *is* stable no longer align across the band.

2 Where the cost actually sits

Rewriting a band wire is cheap: a band-sourced pivot CNOT plus `fill_n1` product terms, about **3 gates**. Everything else is the *release*.

A mask slot names band variables. If a variable’s value changes while a live slot names it, that slot’s strip cancels a different product than its inject installed, and the value’s decode is wrong. So `retire_refill` first re-sources every naming slot — inject a replacement, strip the old one — at **two emissions per reference**. At production the band is sized to the value count, so

$$R = \frac{\text{values} \times \text{factors per value}}{\text{band}} = \text{factors per value} \approx 10$$

for the $[2, 2, 2, 3]$ plan with 50% jitter. One refresh is therefore ~ 20 emissions plus the rewrite — roughly **23 gates against ~ 112 gates per fold**. At **epoch: 5** that is $\sim 2.1\%$ of the circuit for 6.3 turnovers, and buying 12 turnovers the same way would cost $\sim 4\%$.

The same arithmetic says why one cannot simply *wait* for a free variable: unreferenced variables occur with probability $e^{-R} \approx e^{-10}$. The free list is empty, always.

3 The idea: the release is already being paid for

`swap_refresh` retires mask terms at *every* fold — that churn exists to stop time-invariant masks from cancelling in a fold’s before/after XOR, and its cost is already in the budget. Those retirements pick their victims at random. **Point them somewhere.**

A rolling set D of band variables is declared *draining*:

1. **Excluded from every fresh draw** (`draw_slot`, `inject_avoiding_var`), so a member’s reference count can only fall. This is the load-bearing half — without it the count random-walks and never reaches zero.
2. **Preferred by retirement steering.** When `swap_refresh_side` picks which slot of a value to retire, and when `resource` picks which value and slot, a slot naming a member wins over the ordinary pick.
3. **Rewritten on reaching zero references**, then swapped out of D for a fresh variable.

Steering changes *which* slot each retirement takes, not *how many* retirements happen. The release phase does not get cheaper — it disappears, absorbed into work the fold was doing regardless. At $n = 128$, **91.8%** of all retirements land on a draining variable (36,357 of 39,598).

Why the arithmetic comes out to nearly zero

Deleting the **epoch** channel returns its release emissions; adding one retirement side per fold spends a fraction of them. Ledger counters, same seed, $n = 128$:

counter	old	new (3 sides)	Δ
injected	27,616	9,066	−18,550 (the old migrated)
swapped	31,678	39,598	+7,920 (one side $\times$ folds)
rewrite events	1,618	3,768	+2,150
turnovers	6.32	14.72	2.3$\times$
gates	884,408	886,695	+0.3%

The epoch releases were 18,550 injects *and* 18,550 matching strips — 37,100 emissions. One extra side per fold is 15,840. The surplus is what buys the extra turnovers; the residue is the +0.3%.

4 Mechanism details

Both retirement channels are steered. `swap_refresh_side` retires base-degree slots only, by design. But a third of all references live in the degree-3 tower term, and a member held

there would never come free — the tail alone would cap the turnover rate. So steering drops the degree filter, and **resource** (which can reach *any* value) is steered as well. The replacement is always drawn at the RETIRED slot’s degree, so the per-value degree multiset — the mask plan, hence the piling-up commitment — is preserved exactly. A test pins this, because plan drift would move the security claim without moving anything that reports it.

Cheapest slot first. Retirement cost is not flat: a base-degree slot emits one g57, while a degree-3 tower term is a 3-control conjunction the ladder re-spells into ~ 4 gates. Steering therefore takes the lowest-degree candidate slot. This never blocks a tower reference — when a variable’s remaining references are all in tower terms, those are the only candidates — it just stops paying tower prices for work a base term can do. Measured worth: 1.3%.

The rewrite is shared. `rewrite_var` is factored out of `retire_refill` and used by both channels, so they put statistically identical material into the band and an A/B between them measures the *schedule*, not the algebra. Its channel split is unchanged from the symmetric-ports revision: the linear pivot is BAND-SOURCED ONLY (a carrier-sourced pivot is a verbatim linear copy of a masked payload-era state into the band — the measured exact-window leak), while the product terms readmit carriers at `refill_data%`. The rewrite is a SHEAR, $b \oplus = \delta$ rather than $b := \delta$, so balance carries over from the old value for free and each turnover raises the degree instead of resetting it.

Timing. Rotation fires at fold boundaries only. The ladder borrows band wires as scratch mid-chain and restores them at the end of a fragment, so a rewrite landing between a borrow and its restore would corrupt the chain.

$\triangle$ **The schedule must not become legible.** This is a *rolling* set with random membership and one-at-a-time replacement, NOT a fixed partition swept in waves. Disjoint waves would print “these $|D|$ wires stopped being read together, then were all written” — exactly the signal the shuffled `retire_queue` exists to avoid on the epoch path. Set size is $\min(12 \times \text{sides}, \text{band}/6) = 42$ at $n = 128$: large enough that steering finds a candidate, small enough to leave the draw pool comfortable.

5 The one invariant

Nothing may name a variable that is being rewritten. A stale reference count would let a live variable be overwritten, and the failure is silent — the circuit still evaluates, it just computes the wrong thing. Guards, in order of strength:

- `rewrite_var` asserts **unconditionally** (not `debug_assert`) that `var_refs[var] == 0`. Every passing build is evidence.
- `var_refs` is maintained per band VARIABLE, not per wire. Slots name variables and rolling moves variables between wires, so a wire-indexed count would be wrong the moment `--prod-roll` fires. (`add_refs/drop_refs` now take the holding value and maintain a `var_holders` reverse index; both were distributed-mode no-ops before.)
- The mechanism test recounts `var_refs` from the live slots after a build and checks `strip_all` leaves none behind.

- Every exactness test in the swap-refresh group runs at the production rate with a live drain set — band variables are rewritten mid-body while masks are drawn and retired around them.
- `gen_sandwich_gadget`’s permanent forward *and reverse* verify.

6 Measured

$n = 128$, same seed, 7,920 source gates, 256 values, 256 band variables, 512 wires. Baseline is the 2026-08-20 stream (PROD_SWAP=2 PROD_EPOCH=5 PROD_DRAIN=0). All arms forward+reverse verified; 357/357 tests pass.

sides	turnovers	gates	vs baseline	steering hit rate
baseline	6.32	884,408	—	—
2	10.06	825,472	−6.7%	79.6%
3 (shipped)	14.72	886,695	+0.3%	91.8%
4	18.62	940,240	+6.3%	95.8%

Two sides — the *unchanged* retirement rate — already beats the old channel on both axes: 10.1 turnovers against 6.3, at 6.7% *fewer* gates. Everything above that is bought.

△ **Do not extrapolate the table.** Cost per side is not flat: raising the rate pushes an increasing share of retirements onto the expensive degree-3 tier, so the marginal side gets steadily worse. The knee is at 3.

Two corrections to the a-priori estimates

1. **The baseline is ~884k gates, not 1.687M** — ~112 gates per fold, not 213. The larger figure predates the 2026-08-22 peephole work.
2. **Emissions are not uniform in cost.** Pricing every retirement at one g57 underestimated the degree-3 tier by ~4×. Steering blind to degree cost +7.7%; the cheapest-slot rule recovered 1.3% of that.

In the other direction, turnovers per side came in *higher* than predicted (~4.9 vs 3.1), because sides 2 and up select the VALUE through `drain_pick` rather than being confined to the fold’s own operands.

7 Knobs

setting	effect
PROD_SWAP= n	retirement SIDES PER FOLD. Default 3. 2 = the 2026-08-20 stream, 0 restores Gray.
PROD_EPOCH= 5	restores the old release-paying channel. Both double-pays.
PROD_DRAIN= k	drain-set size; 0 isolates the raised rate as its own arm.

$\triangle$ **swap_refresh changed meaning** — it was a flag, tested only > 0 ; it is now a count. `PROD_SWAP=1` no longer means what it used to.

Read `turnovers=` in the `[prod]` report line, not the configured rate. The line also carries `drained=` (rewrites completed) and `steered=` (retirements that landed on a drain member — if this is far below `swapped=`, steering is stalling and the rate is being wasted).

8 Scope

This does not touch the gauntlet’s linear correlations, and cannot. Any correct refresh must emit its own delta, and that delta’s flip bit is exactly the correction a flip-trace adversary needs: inject m_{old} , patch $m_{\text{old}} \oplus m_{\text{new}}$, strip m_{new} telescope. More generally, in an XOR-target gate set every wire value is affine in (init, flips), and the decode is $v = c \oplus \sum_i m_i$ with every m_i emitted by its own gate — so additive masking is affinely transparent by construction. That is why the unprotected control, the secret-share arm, and the production gadget all scored 64 on `xtrace`. The lever there is the decode (see `ENCODED_CHAIN_DESIGN.md`), not the band.

The drain set addresses the `FUNCTION-LIFETIME` axis only. It is a real axis — the elbow lands on the band size, $46 \rightarrow 45$ recovered, $256 \rightarrow 254$ — but it is not the trace axis.

Not yet measured. The write census wants re-running: `band = value count` was chosen so the band and carrier per-wire write distributions coincide (185/452/847 against 180/428/848 at $n = 128$), and that was tuned at the old refresh rate. Raising the rate changes band write rates materially, and homogeneity is what stops a windowed census from isolating the band. `flip_match` and `segment_deduce` are also unre-run.